El Pipeline Completo de IA

Banderazo inicial del proyecto final (10 de noviembre – 15 de enero)

Laboratorio de Inteligencia Artificial

De ejercicios aislados a un **sistema completo y verificable**

- Durante el semestre construimos ladrillos individuales: agentes, redes neuronales, espacios latentes, vectores semánticos y auditoría de tablas CSV.
- Un modelo aislado no es un proyecto: un proyecto es un **pipeline reproducible** desde el dato crudo hasta la autopsia de sus errores.
- Hoy damos el banderazo de salida: tenemos **dos meses** (cierre: **15 de enero**) para diseñar, construir y defender un pipeline completo.

1. Datos y Supervisión

- Auditoría de CSV y limpieza.
- Anomalías (Benford, IQR).
- Regresión y Redes Neuronales.
- *Early stopping* e hiperparámetros.

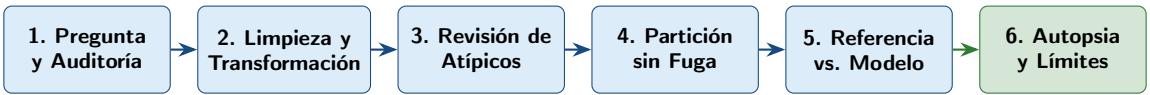
2. Espacio Latente

- Emergencia y autoorganización.
- Mapas de Kohonen (SOM).
- Autoencoders clásicos.
- Modelos generativos (VAE).

3. Texto y Contexto

- Embeddings semánticos.
- Similitud coseno y k-NN.
- Fragmentación y RAG.
- Auto-atención en Transformers.

Regla de oro del proyecto: no hay que meter todos los modelos a la fuerza; hay que elegir la ruta adecuada para una pregunta clara.



- Si falla la Etapa 1 o 2, el modelo aprenderá basura (*garbage in, garbage out*).
- Si falla la Etapa 4, el resultado será un espejismo por fuga de información.
- Si falta la Etapa 5 o 6, no sabremos si el modelo aporta valor real o dónde se rompe.

Ruta A — Tabular y Predicción Supervisada: CSV propio o de simulación (energía, demanda, sensores del robot). Limpieza documentada, detección de atípicos con IQR, regresión o red neuronal con *early stopping*, superando a una referencia en datos de prueba.

Ruta B — Representación y Espacio Latente: SOM, Autoencoder o VAE sobre datos multivariados o registros de episodios para descubrir agrupamientos, comprimir dimensiones, interpolar estados o detectar anomalías de reconstrucción.

Ruta C — Texto, Embeddings y Sistema RAG: Corpus de documentos reales del equipo, comparación de estrategias de *chunking*, base vectorial con similitud coseno y evaluación rigurosa de aciertos y fallos de recuperación.

```python
def limpiar_y_transformar(filas_crudas):
    limpios = []
    for fila in filas_crudas:
        try:
            equipos = float(fila["equipos_activos"])
            consumo = float(fila["consumo_kwh"])
        except ValueError:
            continue  # Celda vacia o texto invalido
        if equipos < 0 or consumo <= 0:
            continue  # Regla de dominio: fisicamente imposible
        limpios.append(fila)
    return limpios
```

Distinción clave para su proyecto

Un valor **imposible** (ej. -5 equipos activos) se corrige o descarta por regla de dominio. Un valor **extremo pero posible** (marcado por IQR) se investiga antes de decidir si se excluye.

Fuga temporal (incorrecto)

Mezclar al azar días de diciembre en el entrenamiento y evaluar en días de octubre.

Consecuencia: el modelo usa pistas del futuro para «adivinar» el pasado.

Fuga por columna (incorrecto)

Predecir `consumo_kwh` incluyendo como entrada `costo_factura` (que se calculó multiplicando el consumo por la tarifa).

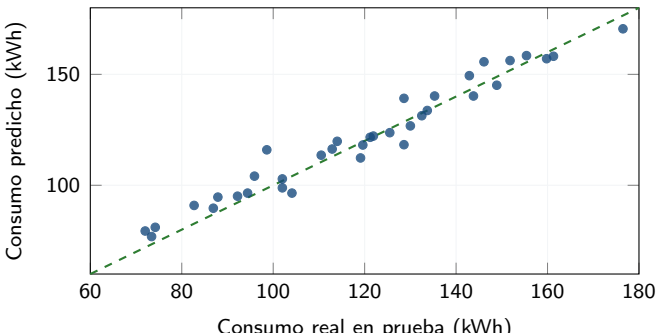
Consecuencia: error casi cero en papel, modelo inútil en la realidad.

Pregunta de control: ¿todas mis variables de entrada X existen *antes* del momento de predecir?



Por qué exigir un baseline

- Decir «mi modelo tiene MAE de 4.83» no significa nada por sí solo.
- Compararlo contra predecir la media del entrenamiento (MAE 22.42) demuestra una reducción del **78.5 %** del error en datos no vistos.



Qué buscamos en esta gráfica

- La línea punteada es la predicción perfecta ($\hat{y} = y$).
- Inspeccionamos los 3 puntos más alejados de la diagonal: ¿qué ocurrió en esas fechas?
- En la calificación final vale más explicar un fallo con rigor que esconderlo.

Hito	Plazo	Entregable verificable
1. Banderazo y Datos	Nov (Sem. 1–2)	Acta de proyecto (`plantilla_propuesta.md`) + carga y auditoría del dataset crudo (Etapas 1 y 2).
2. Limpieza y Referencia	Fines Nov / Dic	Reglas de limpieza, análisis de atípicos, partición sin fuga y métrica del *Baseline* (Etapas 3 y 4).
3. Modelo e Iteración	Diciembre	Entrenamiento del modelo principal, ajuste de hiperparámetros y comparación contra referencia (Etapa 5).
4. Autopsia y Entrega	**15 de enero**	Análisis de los peores errores en prueba, gráficas finales, límites del estudio y defensa (Etapa 6).

Consejo práctico

Un pipeline sencillo que corre completo de la Etapa 1 a la 6 en noviembre es mucho mejor que un modelo gigante que no llega a evaluarse en enero.

- **Claridad de la pregunta y calidad de los datos (25 %):** auditoría explícita de faltantes, tipos, rangos y justificación de qué se conserva o descarta.
- **Diseño sin fuga y referencia justa (25 %):** separación correcta de entrenamiento/prueba y un *baseline* bien calculado.
- **Implementación y comprensión del modelo (25 %):** código limpio, reproducible y capaz de explicarse línea por línea sin cajas negras mágicas.
- **Autopsia de errores y honestidad técnica (25 %):** inspección de fallos concretos y distinción clara entre capacidad predictiva y causalidad.

1. Ejecutar el generador y el pipeline base de referencia:

   `python3 generar_datos_ejemplo.py` y `python3 pipeline_base.py`
2. Inspeccionar las 6 etapas en consola y el archivo generado `predicciones_prueba.csv`.
3. Hacer un experimento rápido en `pipeline_base.py`: quitar `equipos_activos` de `ENTRADAS` y medir cuánto sube el MAE en prueba.
4. Abrir `plantilla_propuesta.md` y redactar el **Acta de Inicio** de su propio proyecto (Ruta A, B o C).

Pregunta clara $\rightarrow$ Datos auditados $\rightarrow$ Referencia $\rightarrow$ Modelo $\rightarrow$ Autopsia

Hoy empieza la construcción de su proyecto final.
Meta: llegar al **15 de enero** con un pipeline completo, reproducible y defendible.